

The Router-Governance Benchmark

A router benchmark must not primarily measure answer quality. It must measure whether the router chose the right **epistemic mode** at the right moment — preventing degeneration *without needlessly blocking everything*. Not "was the answer nice?" but "was the policy decision correct, and did it stop a polluted state update?". Three phases, increasing realism; every number below is pulled live from the committed sources.

Phase 1 · Deterministic fixture benchmark (no LLM)

80 synthetic DesiReport fixtures across 8 classes (A clean · B missing-state · C missing-user-state · D invalidated · E open-conflict · F wrong-frame · G stale/retrieval · H over-block-control), each with an expected mode, verifier requirement and update permission. Seven baselines compete.

baseline	mode_acc	verif_rec	blk_rec	enforce	overblock
B0_no_router	0.10	0.00	0.00	0.00	0.00
B1_always_normal	0.10	0.00	0.00	0.00	0.00
B2_always_retrieval	0.19	1.00	1.00	1.00	0.00
B3_always_state_slice	0.21	0.00	0.00	0.00	0.00
B4_always_guarded	0.25	1.00	1.00	1.00	1.00
B5_simple_threshold	0.71	1.00	1.00	1.00	0.25
B6_desi_router	1.00	1.00	1.00	1.00	0.00

Read the **trade-off**, not the perfect row (desi_router scores high partly by construction — it implements the spec the labels encode; stated openly in cases.py). The load-bearing comparisons hold inputs constant: **desi_router vs always_guarded** — identical safety (block-recall 1.00) but over-blocking **0.00 vs 1.00**; and **desi_router vs simple_threshold** — both see the same risk_scores, yet the ordered most-cautious-first policy beats a single 0.5 threshold (mode-acc 1.00 vs 0.71). That delta is not circular.

Phase 2 · Replay against the real ablation artefacts (no LLM)

For each of 28 (model, condition) points from the committed ablation JSONs (claude-sonnet-4.5, gpt-4o, granite-4.1-8b), label it DEGENERATE/CLEAN from its *measured* metrics, map the condition to the situation the router would see, run select_mode, and check concordance: does the router protect ■ degeneration was measured?

pass	clean	no state/retrieval	open conflict	plausible wrong slice	overall
signaled	1.00	1.00	1.00	1.00	1.00
unsigned	1.00	1.00	1.00	0.00	0.86

Structural risks (no usable state → retrieval, an open conflict) are caught in **both** passes. A plausible-wrong slice is caught only when DESi/Layer-9 **signals** it (low confidence/recall/wrong-frame): concordance 1.00 signaled vs 0.00 unsigned — the discordant points are exactly C_wrong_slice + G_neutral_irrelevant. **The router's protection against undetectable wrong state equals its input signal, no better.** That dependency, not a headline number, is the Phase-2 result.

Phase 3 · Live closed-loop (real models)

8 scenarios × 2 arms × 2 models (claude-sonnet-4.5, granite-4.1-8b, temperature 0). Both arms see the same facts; no_router gets a neutral prompt, desi_router gets the governance status (guarded preprompt) + a post-answer verifier gate. Outcomes measured with the router's own verifier.

arm	invalid-reuse	critical_rate	pollution_rate
no_router	0.00	0.19	0.19
desi_router	0.08	0.19	0.00

Robust result — the gate prevents state pollution. no_router let **3 polluted updates** through (it closed open conflicts E1/E2 without evidence); desi_router's gate blocked **all** → pollution **0.19** → **0.00**. The live demonstration of the layer's purpose.

Honest twist — the rule verifier's precision is the bottleneck, not the model or the policy. On inspection, every desi_router "critical" flag this run is a **false positive on a correct, cautious answer**: in D2 the governed model correctly picked schema-per-tenant *while flagging the superseded option* (the token-overlap check misread the rejection as reuse); in E1 it correctly **refused to close the conflict** ("I cannot resolve ... evidence for either position is missing") yet was flagged conflict_closure. The run found a real negation-blindness, fixed for the clearest case; structural false positives remain. **Conclusion: gate and policy sound; critical_rate is not yet a trustworthy degeneration signal** — a semantic / NLI verifier (Phase 3.5) is the needed upgrade. Small N; temperature-0 is not fully deterministic on OpenRouter → directional, not a leaderboard.

Phase 4 · multi-turn relapse · Phase 3.5 · semantic verifier

Inject an invalidated claim, then ask → neutral double-check → a later tempting question. Relapse = the bad claim is reused in turn 2/3. The rule verifier over-counts (it cannot tell quarantine from relapse), so Phase 3.5 adds an LLM-as-judge (adopts/rejects/absent) for measurement only — the runtime gate stays the deterministic rule verifier.

arm	rule relapse	semantic relapse (corrected)
no_router	1.00	0.67
desi_router	1.00	0.00

Of 10 rule-flagged turns the judge confirmed only 3 as genuine adoption — 7 were rejection/quarantine false positives. Corrected: **the persisted guarded preprompt drives genuine relapse to 0.00, while no_router actually relapses (0.67)** — the ablation's "injected claims persist and relapse" effect, suppressed by the router. The lasting design lesson: **deterministic gate at runtime, semantic judge for evaluation**. Small N; the judge is itself an LLM — directional.

What the benchmark does and does not claim

- **Shown:** the DESi router is *selectively* safe — it reaches always_guarded's gate safety at 0.00 over-blocking, and beats a naive threshold on the same signals.
- **Shown:** the router routes protectively exactly where the ablation measured degeneration — for structural risk unconditionally, for plausible-wrong state as far as DESi signals it.
- **Shown (live):** the verifier gate prevents polluted persistent updates on real outputs.
- **NOT claimed:** that the guarded preprompt makes the model answer better — both arms answered correctly; the apparent difference is verifier precision, not model behaviour.
- **NOT claimed:** metadata governance is a recall effect — B ≈ E stands (from the ablation).
- **Shown (Phase 4 + 3.5):** the persisted guard drives genuine multi-turn relapse to 0.00 vs 0.67 — but only the semantic judge makes that measurable; the rule verifier is a gate, not a measure.

Reproduce: `python -m desi_router.governance.benchmark.run ...benchmark.replay ...benchmark.live_loop (key via env, never committed) · pytest tests/router_governance -q (39 tests).`